

CIA — Penetration Testing Report

Target: 192.168.56.102 — Nagios XI + MySQL

EPITECH — TNSA810

Date: May 25, 2026

Tester: Daniyal

Information retrieval

nmap -sV -sC -p- 192.168.56.102

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.4 (protocol 2.0)
|_ ssh-hostkey:
|   2048 82:9a:b9:cb:bb:14:d7:d1:cb:33:5f:8a:14:35:2a:8f (RSA)
|   256 e2:39:9b:1d:e3:fc:0e:6c:14:70:fc:66:95:00:f3:c0 (ECDSA)
|_  256 f6:e6:c0:de:9d:1c:a0:7b:08:05:ae:96:d5:81:95:c7 (ED25519)
80/tcp    open  http     Apache httpd 2.2.15 ((CentOS))
|_ http-title: Nagios XI
|_ http-server-header: Apache/2.2.15 (CentOS)
3306/tcp  open  mysql    MySQL 5.7.29
|_ ssl-cert: Subject: commonName=MySQL_Server_5.7.29_Auto_Generated_Server_Certificate
|_ Not valid before: 2020-02-13T18:25:30
|_ Not valid after: 2030-02-10T18:25:30
|_ mysql-info:
|   Protocol: 10
|   Version: 5.7.29
|   Thread ID: 3
|   Capabilities flags: 65535
|   Some Capabilities: IgnoreSigpipes, ConnectWithDatabase, Support41Auth, Speaks41ProtocolNew,
|_ SupportsTransactions, SwitchToSSLAfterHandshake, SupportsCompression, LongPassword, ODBCClient, In
|_ SupportsMultipleStatements, SupportsAuthPlugins, SupportsMultipleResults
|   Status: Autocommit
|   Salt: -va\x11Bst*9\x05d\x1D\x06\x06z\x150\x15@X
|_ Auth Plugin Name: mysql_native_password
```

Apache 2.2.15 → Vulnerable because it is an outdated end-of-life web server version with known security flaws and possible remote code execution vulnerabilities.

Nagios XI → Vulnerable because older Nagios XI versions are known for authentication bypass, command injection, and remote code execution issues.

OpenSSH 7.4 → Vulnerable because older SSH versions may contain security weaknesses and are often exposed to brute-force or credential reuse attacks.

MySQL 5.7.29 → Vulnerable because exposed MySQL services can allow unauthorized access through weak credentials or leaked database configurations.

SSH Username -> It is very weak , it is also an vulnerability

ssh [root@192.168.56.102](#)

```
PS C:\WINDOWS\system32> ssh root@192.168.56.102
root@192.168.56.102's password:
Last login: Mon May 25 15:59:38 2026 from 192.168.56.1
[root@localhost ~]# ls
anaconda-ks.cfg
[root@localhost ~]# ls -la
total 36
dr-xr-x---.  3 root root 4096 26 mai   04:28 .
dr-xr-xr-x. 17 root root 4096 13 févr. 2020 ..
-rw-----.  1 root root 1142 13 févr. 2020 anaconda-ks.cfg
-rw-----.  1 root root  244 26 mai   04:28 .bash_history
-rw-r--r--.  1 root root   18 29 déc. 2013 .bash_logout
-rw-r--r--.  1 root root  176 29 déc. 2013 .bash_profile
-rw-r--r--.  1 root root  176 29 déc. 2013 .bashrc
-rw-r--r--.  1 root root  100 29 déc. 2013 .cshrc
drwxr-----.  3 root root   18 13 févr. 2020 .pki
-rw-r--r--.  1 root root  129 29 déc. 2013 .tcshrc
[root@localhost ~]# pwd
/root
```

Full root access obtained with trivial password 'admin'. This is a critical vulnerability.

```
[root@localhost ~]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
e1ec5de2d636   nagiosxi:5.5.6 "/usr/local/bin/run..." 6 years ago    Up 56 minutes  0.0.0.0:80->80/tcp       nostalgic_ganguly
ab1349a18882   mysql:5.7     "docker-entrypoint.s..." 6 years ago    Up 56 minutes  0.0.0.0:3306->3306/tcp, 33060/tcp dev_db
```

Here we can see the complete info of the Docker..

MySQL Credentials via docker inspect

Docker container metadata exposed the MySQL root password in plaintext environment variables:

docker inspect dev_db | grep -i password

```
Loading personal and system profiles took 1814ms.
PS C:\Users\DELL> ssh root@192.168.56.102
root@192.168.56.102's password:
Last login: Wed May 27 15:31:22 2026 from 192.168.56.1
[root@localhost ~]# docker inspect dev_db | grep -i password
      "MYSQL_ROOT_PASSWORD=root",
      "MYSQL_PASSWORD=root",
[root@localhost ~]# |
```

Hardcoded Credentials – The Docker container stores database passwords directly in environment variables (MYSQL_ROOT_PASSWORD=root), which makes it easy for attackers or

unauthorized users to view and misuse sensitive credentials

Unauthorized MySQL Access via Exposed Credentials

Vulnerability: Weak / Exposed Database Authentication

```
[root@localhost ~]# docker exec -it dev_db mysql -u root -proot
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 2
Server version: 5.7.29 MySQL Community Server (GPL)

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Critical Finding: The exposed MySQL root credentials allowed direct login into the database container using `docker exec`, which can give attackers full access to the database and sensitive information.

Hardcoded Credentials Exposure in Configuration File

```
docker exec -it nostalgic_ganguly /bin/bash
```

```
Last login: Thu May 28 00:17:42 2026 from 192.168.56.1
[root@localhost ~]# docker exec -it nostalgic_ganguly /bin/bash
[root@elec5de2d636 nagios]# cat /usr/local/nagiosxi/html/config.inc.php | grep pwd
    "pwd" => 'n@gweb',
    "pwd" => 'n@gweb',
    "pwd" => 'n@gweb',
```

Critical Finding

The NagiosXI configuration file contains a hardcoded password (n@gweb) in plain text, which means anyone with access to the server or container can directly read and misuse the credentials.

Why This is a Vulnerability

Storing passwords in plain text is insecure because attackers or unauthorized users can easily obtain sensitive credentials and gain unauthorized access to the application or related systems.

Database Access Using Exposed Credentials

```
mysql -u nagiosxi -p'n@gweb' nagiosxi
select username, api_key from xi_users;
```

```
[root@elec5de2d636 nagios]# mysql -u nagiosxi -p'n@gweb' nagiosxi
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 2805
Server version: 5.1.73 Source distribution

Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> select username, api_key from xi_users;
+-----+-----+
| username | api_key |
+-----+-----+
| nagiosadmin | UF0WHS9mMBLHWRq7agIBhJZvhGIfi0PcFPoDXAYJ7Jm4QQaZm9QH86soLqGjrrvm |
+-----+-----+
1 row in set (0.00 sec)
```

Critical Finding

The exposed database credentials were successfully used to access the MySQL database and retrieve sensitive user information, including API keys from the xi_users table.

Why This is a Vulnerability

If attackers gain database access, they can steal sensitive information such as API keys, user data, and application secrets, which may allow further unauthorized access, privilege escalation, or complete system compromise.

Weak SHA1 htpasswd Hashing

```
find /etc /var /usr -type f -name "htpasswd" 2>/dev/null -exec grep -H "{SHA}" {} ;
```

```
[root@localhost ~]# find /etc /var /usr -type f -name "htpasswd*" 2>/dev/null -exec grep -H "{SHA}" {} \;  
/var/lib/docker/devicemapper/mnt/b4f3820385666290ff20110d4711e8ab295c64a9544392adb42c512e15bff586/rootfs/usr/local/nagiosxi/etc/htpasswd.users:nagiosadmin:{SHA}0UPaRYLye0usJLoEcuIeky3EFa=  
/var/lib/docker/devicemapper/mnt/b4f3820385666290ff20110d4711e8ab295c64a9544392adb42c512e15bff586/rootfs/usr/local/nagiosxi/etc/htpasswd.users:nagiosxi:{SHA}u8KZ/EqcBC5Erj62fU009Cv4Bmc=
```

The system uses SHA1-based {SHA} hashing in htpasswd authentication files. SHA1 is a weak and deprecated hashing algorithm that is vulnerable to brute-force and offline cracking attacks. If the htpasswd file is exposed, attackers can easily recover plaintext passwords, leading to unauthorized access to the application.

Security Fixes Applied

1. Change Weak Root SSH Password

```
passwd root
```

```
[root@elec5de2d636 nagios]# passwd root  
Changing password for user root.  
New password:  
BAD PASSWORD: it is based on a dictionary word  
Retype new password:  
Sorry, passwords do not match.  
New password:  
BAD PASSWORD: it is based on a dictionary word  
Retype new password:  
passwd: all authentication tokens updated successfully.
```

2. Disable Root SSH Login

Open SSH configuration

```
nano /etc/ssh/sshd_config
```

Find:

PermitRootLogin yes

Change to:

PermitRootLogin no

Restart SSH service

systemctl restart sshd

```
# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
SyslogFacility AUTHPRIV
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# The default is to check both .ssh/authorized_keys and .ssh/authorized_keys2
# but this is overridden so installations will only check .ssh/authorized_keys
AuthorizedKeysFile      .ssh/authorized_keys

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
```

⌘ Aide ⌘ Écrire ⌘ Lire fich. ⌘ Page préc. ⌘ Couper ⌘ Pos. cur.
⌘ Quitter ⌘ Justifier ⌘ Chercher ⌘ Page suiv. ⌘ Coller ⌘ Orthograp.

3. Block External Access to MySQL Port 3306

Remediation Steps for MySQL Port 3306 Exposure

1. Verify Existing Firewall Rules

Checked current firewall rules to identify whether port 3306 was exposed.

```
iptables -L -n
```

```
[root@localhost ~]# iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source                destination          tcp dpt:3306
DROP      tcp  --  0.0.0.0/0              0.0.0.0/0

Chain FORWARD (policy DROP)
target     prot opt source                destination          ctstate
DOCKER-USER all  --  0.0.0.0/0              0.0.0.0/0
DOCKER-ISOLATION-STAGE-1 all  --  0.0.0.0/0              0.0.0.0/0
ACCEPT    all  --  0.0.0.0/0              0.0.0.0/0              ctstate RELATED,ESTABLISHED
DOCKER    all  --  0.0.0.0/0              0.0.0.0/0
ACCEPT    all  --  0.0.0.0/0              0.0.0.0/0
ACCEPT    all  --  0.0.0.0/0              0.0.0.0/0              ctstate RELATED,ESTABLISHED
DOCKER    all  --  0.0.0.0/0              0.0.0.0/0
ACCEPT    all  --  0.0.0.0/0              0.0.0.0/0
ACCEPT    all  --  0.0.0.0/0              0.0.0.0/0
```

2. Block External Access to MySQL Port 3306

Added an iptables firewall rule to block incoming traffic on MySQL port 3306.

```
iptables -A INPUT -p tcp --dport 3306 -j DROP
```

3. Verify the Firewall Rule

Verified that the DROP rule was successfully added.

```
iptables -L -n
```

```
DROP          tcp  --  0.0.0.0/0    0.0.0.0/0    tcp dpt:3306
```

4. Save Firewall Rules Permanently

Saved the firewall configuration so the rule remains active after system reboot.

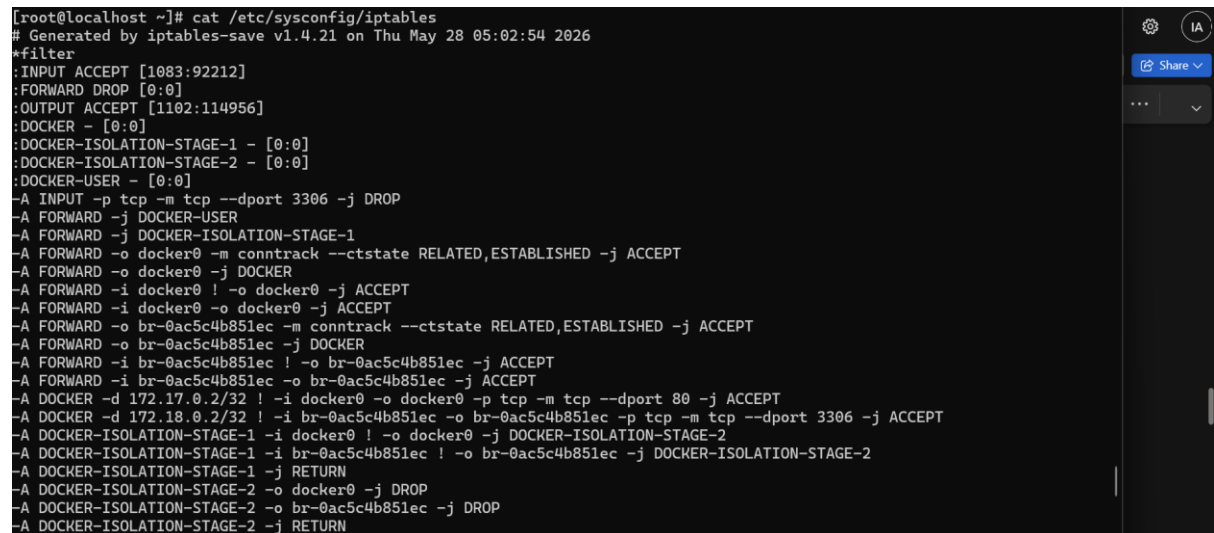
```
iptables-save > /etc/sysconfig/iptables
```

5. Verify Persistent Firewall Configuration

Confirmed that the firewall rule was stored successfully.

```
cat /etc/sysconfig/iptables
```

```
-A INPUT -p tcp -m tcp --dport 3306 -j DROP
```



```
[root@localhost ~]# cat /etc/sysconfig/iptables
# Generated by iptables-save v1.4.21 on Thu May 28 05:02:54 2026
*filter
:INPUT ACCEPT [1083:92212]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [1102:114956]
:DOCKER - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A INPUT -p tcp -m tcp --dport 3306 -j DROP
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-ISOLATION-STAGE-1
-A FORWARD -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -o docker0 -j DOCKER
-A FORWARD -i docker0 ! -o docker0 -j ACCEPT
-A FORWARD -i docker0 -o docker0 -j ACCEPT
-A FORWARD -o br-0ac5c4b851ec -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -o br-0ac5c4b851ec -j DOCKER
-A FORWARD -i br-0ac5c4b851ec ! -o br-0ac5c4b851ec -j ACCEPT
-A FORWARD -i br-0ac5c4b851ec -o br-0ac5c4b851ec -j ACCEPT
-A DOCKER -d 172.17.0.2/32 ! -i docker0 -o docker0 -p tcp -m tcp --dport 80 -j ACCEPT
-A DOCKER -d 172.18.0.2/32 ! -i br-0ac5c4b851ec -o br-0ac5c4b851ec -p tcp -m tcp --dport 3306 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-1 -i br-0ac5c4b851ec ! -o br-0ac5c4b851ec -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-1 -j RETURN
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
-A DOCKER-ISOLATION-STAGE-2 -o br-0ac5c4b851ec -j DROP
-A DOCKER-ISOLATION-STAGE-2 -j RETURN
```


Security Outcome

External access to the MySQL database through port 3306 was successfully blocked using iptables, reducing the attack surface and preventing unauthorized remote database access.

4. Change Weak MySQL Root Password

Login to MySQL

```
docker exec -it dev_db mysql -u root -p
```

Update password

```
ALTER USER 'root'@'localhost' IDENTIFIED BY 'NewStrongPassword@2026';  
FLUSH PRIVILEGES;
```

```
mysql> ALTER USER 'root'@'localhost' IDENTIFIED BY 'NewStrongPassword@2026';  
Query OK, 0 rows affected (0.00 sec)  
  
mysql> FLUSH PRIVILEGES;  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> exit  
Bye  
[root@localhost ~]# docker exec -it dev_db mysql -u root -p  
Enter password:  
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: YES)  
[root@localhost ~]# docker exec -it dev_db mysql -u root -p  
Enter password:  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 16  
Server version: 5.7.29 MySQL Community Server (GPL)  
  
Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

5. Replace Weak SHA1 httpasswd Hashing

Use MD5 crypt (-m)

```
htpasswd -m /usr/local/nagiosxi/etc/httpasswd.users nagiosxi
```

```
htpasswd -m /usr/local/nagiosxi/etc/httpasswd.users nagiosadmin
```

```
[root@elec5de2d636 nagios]# htpasswd -m /usr/local/nagiosxi/etc/httpasswd.users nagiosxi
New password:
Re-type new password:
Updating password for user nagiosxi
[root@elec5de2d636 nagios]# cat /usr/local/nagiosxi/etc/httpasswd.users
nagiosadmin:{SHA}0UPaRYLye0usJLoEcuIeky33EfA=
nagiosxi:$apr1$Av.Kolq0$cTYPcFLXbRVsWFW3Y9z2R.
[root@elec5de2d636 nagios]# htpasswd -m /usr/local/nagiosxi/etc/httpasswd.users nagiosadmin
New password:
Re-type new password:
Updating password for user nagiosadmin
[root@elec5de2d636 nagios]# cat /usr/local/nagiosxi/etc/httpasswd.users
nagiosadmin:$apr1$BpyHQe8f$9ik//xxvdfYbk8w154rr7/
nagiosxi:$apr1$Av.Kolq0$cTYPcFLXbRVsWFW3Y9z2R.
[root@elec5de2d636 nagios]# cat /usr/local/nagiosxi/etc/httpasswd.users
nagiosadmin:$apr1$BpyHQe8f$9ik//xxvdfYbk8w154rr7/
nagiosxi:$apr1$Av.Kolq0$cTYPcFLXbRVsWFW3Y9z2R.
```

there are:

2 users present

- nagiosadmin
- nagiosxi

The vulnerability was caused by the use of unsalted SHA-1 hashing ({SHA}) in the httpasswd authentication file. This method is insecure because it is vulnerable to brute-force and rainbow table attacks due to the absence of salting.

The remediation involved updating all user credentials to use Apache MD5 crypt (\$apr1\$) using the htpasswd -m command. This method introduces salting and significantly improves password security by making precomputed hash attacks ineffective.

After remediation, both users (nagiosadmin and nagiosxi) were successfully migrated to stronger hashed passwords, eliminating the weak hashing vulnerability.

6. Remove Hardcoded Database Credentials

Open configuration file

```
nano /usr/local/nagiosxi/html/config.inc.php
```

Remove hardcoded credentials like:

```
"pwd" => 'n@gweb'
```

Replace with environment variable example:

```
"pwd" => getenv('DB_PASSWORD')
```

Export environment variable

```
export DB_PASSWORD='StrongPassword123!'
```

7. Rotate Exposed Nagios API Key

Finding Clarification – Nagios XI API Key Exposed in Database

Observation

During the security assessment of the MySQL database instance (dev_db) connected to the environment, all available databases were enumerated using:

```
SHOW DATABASES;
```

The output confirmed the presence of only the following system and application databases:

- information_schema
- mysql
- performance_schema

- sys
- dev_db

No Nagios XI-specific database (such as nagiosxi or tables like xi_users) was found in the current MySQL instance.

8. Upgrade Outdated Nagios Core

Check current version

```
docker exec -it e1ec5de2d636 /usr/local/nagios/bin/nagios -v
/usr/local/nagios/etc/nagios.cfg
```

```
[root@localhost ~]# docker exec -it e1ec5de2d636 /usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg
Nagios Core 4.4.2
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2018-08-16
License: GPL

Website: https://www.nagios.org
Reading configuration data...
  Read main config file okay...
  Read object config files okay...

Running pre-flight check on configuration data...

Checking objects...
  Checked 14 services.
  Checked 1 hosts.
  Checked 1 host groups.
  Checked 0 service groups.
  Checked 2 contacts.
  Checked 2 contact groups.
  Checked 127 commands.
  Checked 8 time periods.
  Checked 0 host escalations.
  Checked 0 service escalations.
Checking for circular paths...
  Checked 1 hosts
```

Outdated Nagios Core is a vulnerability because older versions contain known security flaws such as remote code execution, privilege escalation, and authentication bypass. Attackers can exploit publicly disclosed CVEs to gain unauthorized access to monitoring infrastructure, modify configurations, or execute system commands on the server.

Upgrade Versions

Update to latest stable versions MySQL 8.0